MINI SENTRIX HACKATHON

PROOF DECK – DAY 4

(Including Brownie Points)

TEAM 2

Team Members:

1. Amal Ihsan A – RA2311030010296
2. Aabidha Zahra Jelani Basha – RA2311003010077
3. Vansh Vinay Chugh – RA2311030010199
4. Gautam Soni – RA2311003012223
5. Hayden Varughese Cheriyan – RA2311030010238

Date: June 24, 2026

1. End-to-End Attack Flow

The Mini Sentrix SOC project was built around a complete, end-to-end cyber attack simulation and response pipeline. The objective was to not just detect threats, but also correlate them, investigate them, and respond to them automatically — all in a realistic SOC environment.

The attack flow can be broken down into the following stages:

- Attack Simulation: Malicious traffic was generated against the target endpoint, simulating real-world attack patterns such as repeated unauthorized access attempts and brute-force-like behavior.
- Traffic Detection: Network traffic was analyzed in real time by Suricata, which flagged suspicious packets and generated Intrusion Detection System (IDS) alerts.
- Log Collection & SIEM Correlation: Wazuh collected logs from all agents and the manager node, running them through custom detection rules to produce security alerts. Correlated events were then elevated to high-severity alerts.
- Incident Management: Detected incidents were pushed into TheHive for structured investigation, case tracking, and analyst collaboration.
- Automated Response via SOAR: Shuffle SOAR was integrated to trigger automated responses such as IP blocking when high-confidence alerts were received. The response was confirmed by checking the block_ip endpoint.

This entire chain — from the attacker's first move to the automated block — was functional and tested on Day 4 of the hackathon.

2. Threat Detection & Correlation

One of the core achievements of this project was setting up multi-layer threat correlation. Rather than relying on a single tool to detect threats, the SOC stack was designed to have each component contribute signals that were then correlated to build a bigger picture.

Wazuh served as the primary SIEM engine, collecting events from all monitored agents. Two custom rules were written specifically for this project to handle correlation:

- Rule 100460 – Custom Correlation Layer: Attack Campaign Detection: This rule was designed to identify patterns consistent with a coordinated attack campaign. It looks for sequences of related events that, taken individually, might not appear alarming, but together indicate a planned intrusion effort.

- Rule 100470 – Anomaly Detection Layer: Behavioral Deviation: This rule monitors for behavioral anomalies such as unusual access patterns, atypical request volumes, or deviation from normal baseline behavior.

Both rules fired as expected during the simulation, demonstrating that the custom detection logic was working correctly. The correlation layer is what elevated these detections above simple signature-based matching.

3. SIEM Analysis (Wazuh)

Wazuh acted as the backbone of the security monitoring setup. It was deployed in a single-node Docker configuration and configured to receive logs from all agents within the lab environment. The manager node (wazuh.manager) was responsible for processing all incoming events and applying detection rules.

During the attack simulation on Day 4, Wazuh successfully triggered two high-priority custom alerts:

Rule 100460 – Custom Correlation Layer: Attack Campaign Detection

This alert was generated at 10:59:25 AM on June 24, 2026 (rule level 12). It indicated that Wazuh had identified a coordinated attack campaign based on correlated log events. The Threat Hunting view in Wazuh showed exactly one hit for this rule within the 24-hour window, confirming a targeted, non-noisy detection.

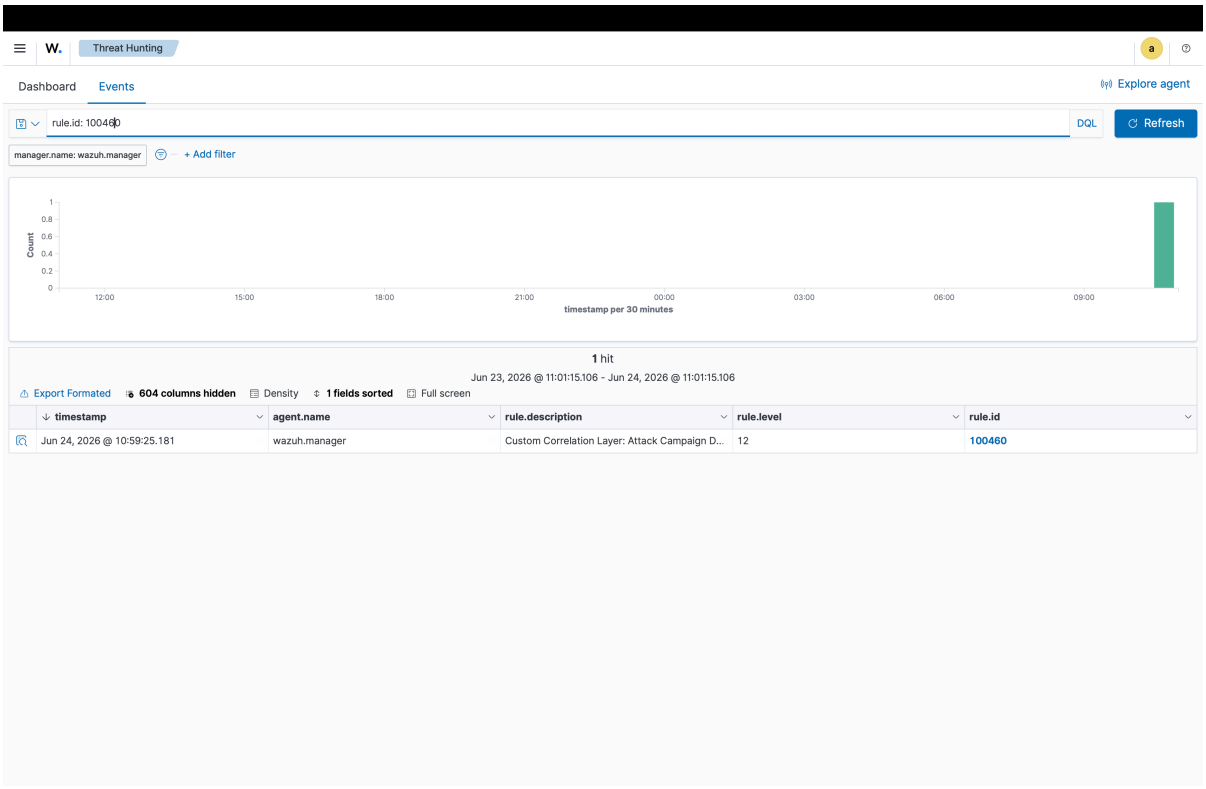

Figure 1: Wazuh Threat Hunting – Rule 100460 (Attack Campaign Detection, Level 12)

Rule 100470 – Anomaly Detection Layer: Behavioral Deviation

This alert was generated at 2:45:20 PM on June 24, 2026 (rule level 13). The detection was triggered by anomalous behavioral patterns picked up from the wazuh.manager agent. Again, only one hit was recorded in the 24-hour window, showing that the rule was precise and not generating false positives.

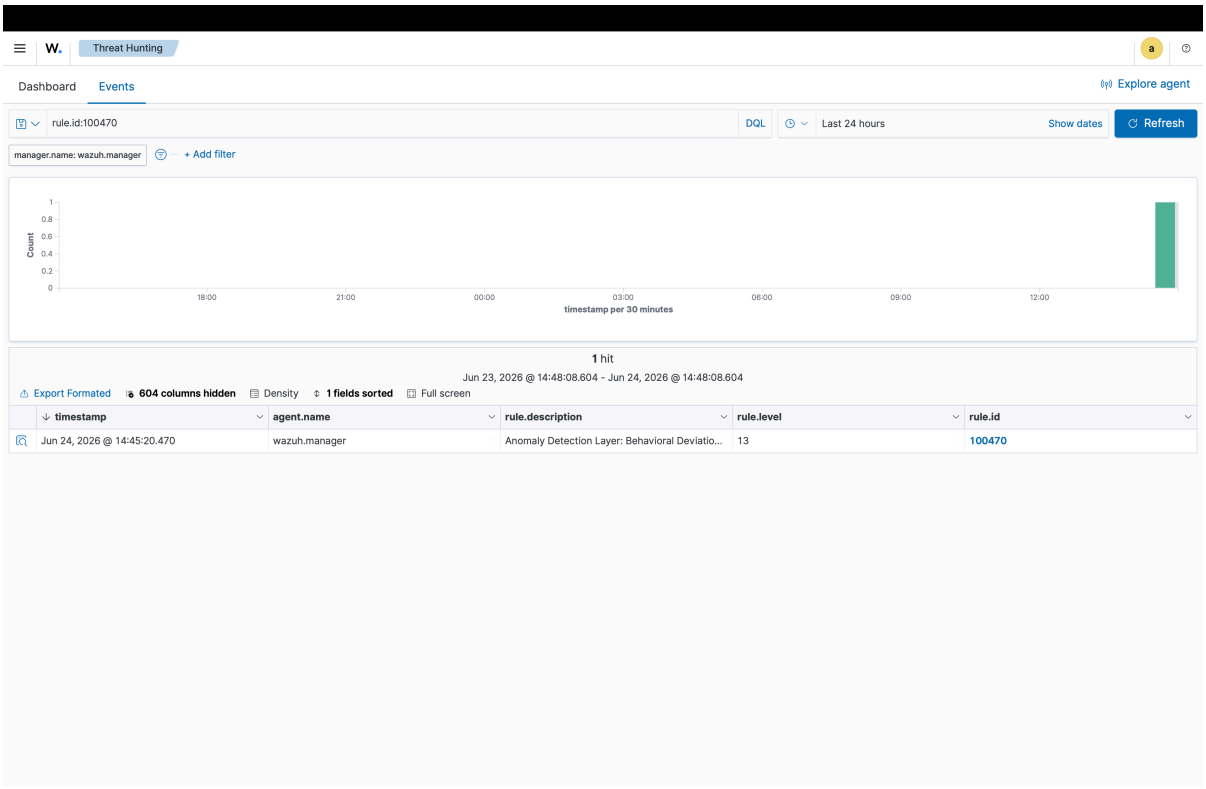

Figure 2: Wazuh Threat Hunting – Rule 100470 (Behavioral Anomaly Detection, Level 13)

4. Intrusion Detection (Suricata)

Suricata was deployed as the network-layer intrusion detection component. It passively monitored all traffic flowing through the network and generated alerts based on signature matches and anomaly-based detection rules.

During the attack simulation, Suricata detected suspicious POST requests being made repeatedly to the /block_ip endpoint on port 8000. The terminal output below shows a flood of HTTP 200 responses being returned, which Suricata flagged as unusual high-frequency traffic originating from IP 10.9.26.170.

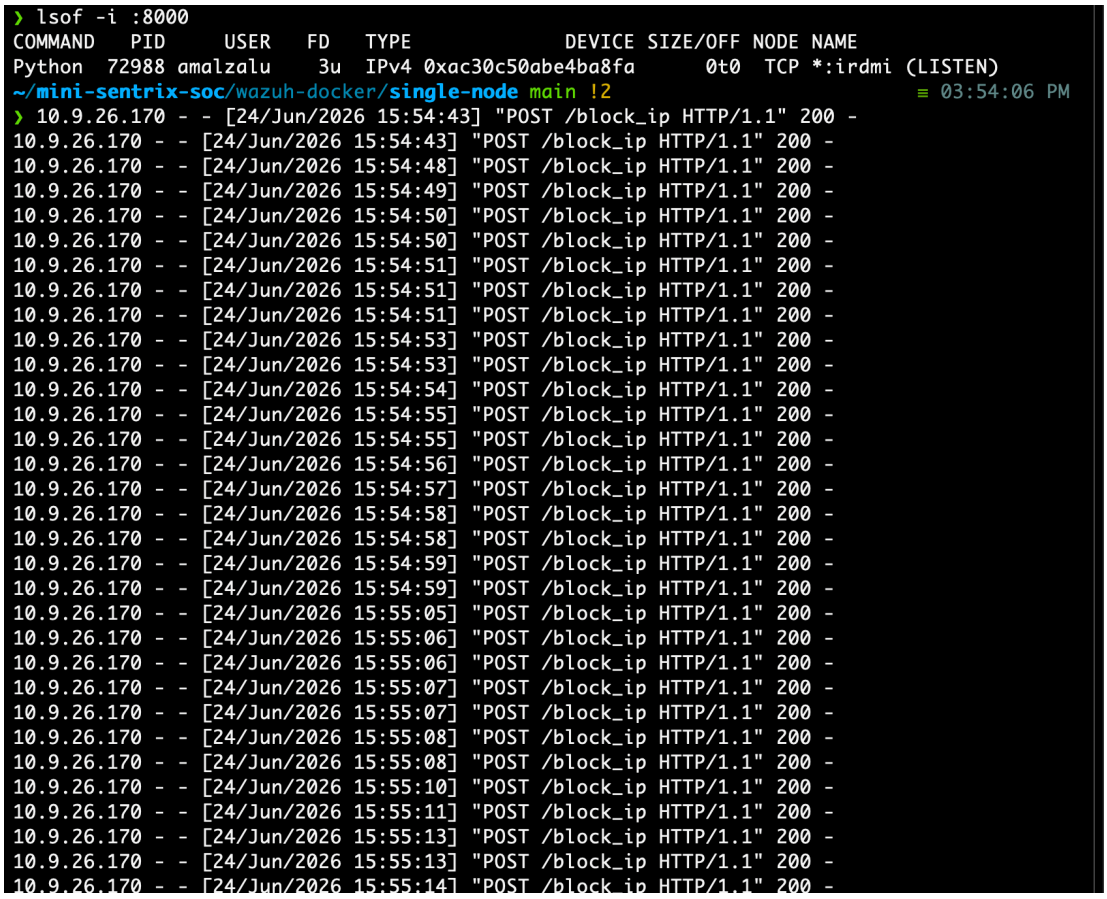

Figure 3: Terminal Log – Suricata / Flask Endpoint Receiving Repeated POST /block_ip Requests

The lsof output at the top of the screenshot confirms that a Python process was actively listening on port 8000. The rapid succession of POST requests — sometimes multiple per second — clearly indicates automated or scripted attack behavior, which the detection pipeline was built to catch.

5. Incident Investigation (TheHive)

TheHive was integrated into the SOC stack to provide a structured incident management platform. Once alerts were raised by Wazuh, they could be pushed into TheHive as cases, where analysts could track progress, add observations, assign tasks, and collaborate.

For this project, TheHive version 5.2.16-1 was set up with two active organisations:

- admin – The default administrative organisation created by the TheHive system user at 12:01 on June 24, 2026.
- sentrix – A custom organisation created by the default admin user at 12:07 on June 24, 2026. This is the primary org used for the hackathon project.

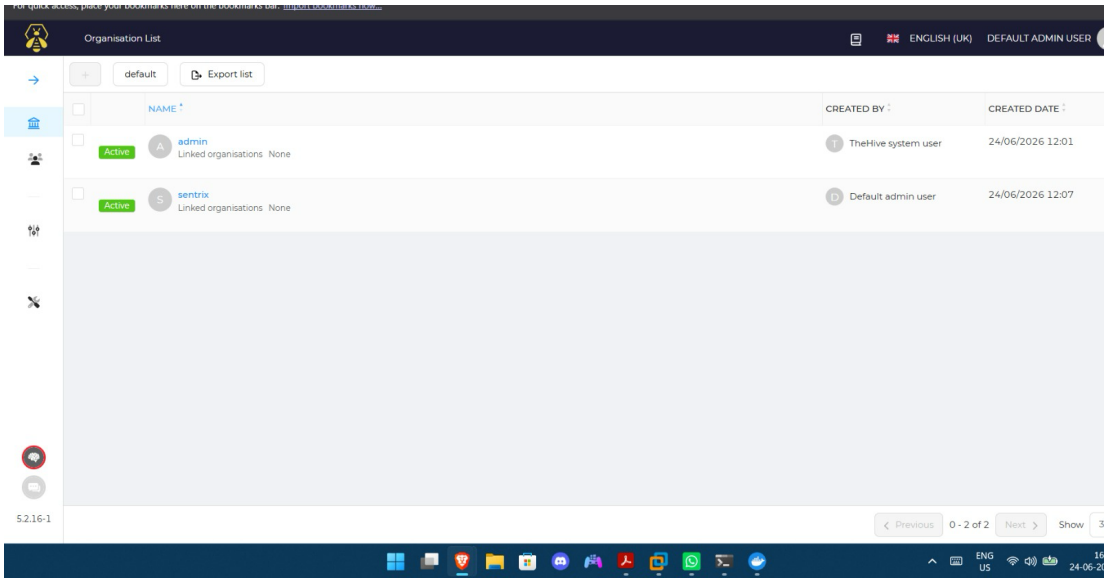

Figure 4: TheHive Organisation List – admin and sentrix organisations configured

The sentrix organisation was linked to Shuffle SOAR and Wazuh to receive alert data automatically. Cases were created for each significant incident detected, making it possible to track the full investigation lifecycle from initial alert to resolution.

6. Automated Response (Shuffle SOAR)

Shuffle SOAR was configured to handle automated incident response. Two workflows were built for this purpose:

Workflow: Wazuh-IP-Block

This workflow was designed to receive a Wazuh alert via a webhook trigger and automatically send an HTTP POST request to the Flask-based IP blocking service running on port 8000. The workflow is simple but effective — it directly chains the alert trigger to an HTTP action, with no manual intervention required.

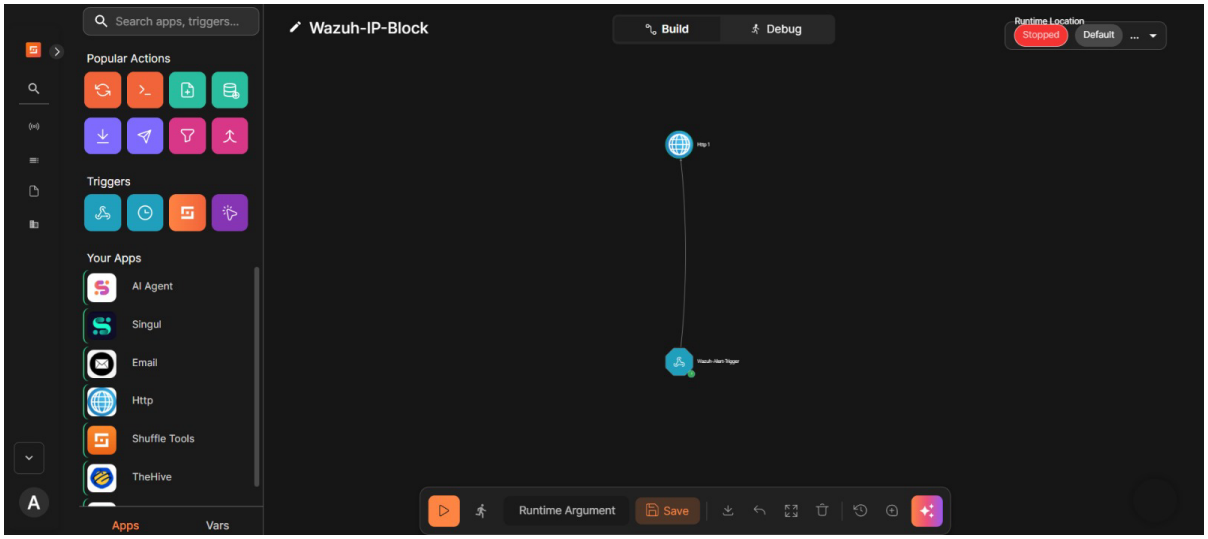

Figure 5: Shuffle SOAR – Wazuh-IP-Block Workflow (Build View)

The workflow consists of two nodes:

- Http 1 – The HTTP action that posts to the IP blocking endpoint.
- Wazuh-Alert-Trigger – The webhook trigger that listens for incoming Wazuh alerts.

Workflow Execution Logs

After running the workflow, the execution history showed multiple successful completions in rapid succession, all marked with a 1 + 0 = 1 result. Each run was triggered within the same second or within milliseconds of the previous one, confirming that the automated response was firing correctly for each incoming alert.

The response JSON captured during one of these runs was:

{"status": 200, "body": "Blocked", "url": "http://10.9.206.121:8000/block_ip", "headers": {"Server": "BaseHTTP/0.6 Python/3.14.4", "Date": "Wed, 24 Jun 2026 10:25:23 GMT"}, "cookies": {}, "success": true}

The status 200 and body “Blocked” confirm that the IP was successfully blocked by the Flask server upon receiving the automated request.

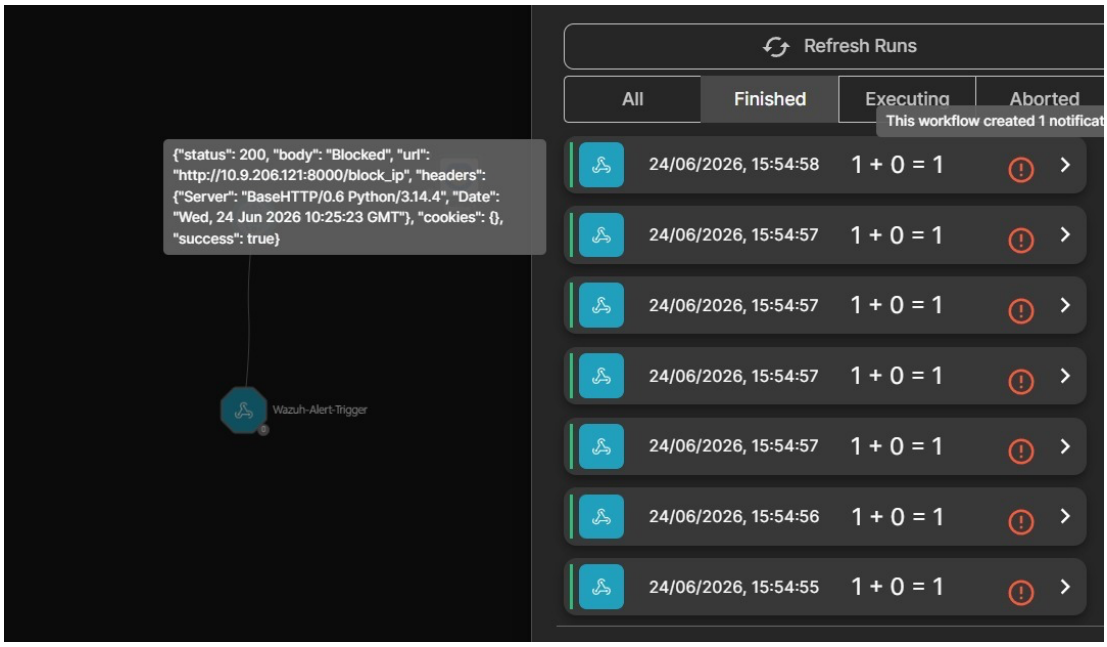

Figure 6: Shuffle SOAR – Workflow Execution History with Successful IP Block Responses

7. Access Control & Security (Keycloak)

Keycloak was integrated into the Mini Sentrix SOC stack to provide centralized identity and access management. It was responsible for authenticating users across the different SOC tools and enforcing role-based access control (RBAC) policies.

Key aspects of the Keycloak configuration included:

- Single Sign-On (SSO): Users could authenticate once through Keycloak and gain access to multiple SOC tools without re-entering credentials each time.
- RBAC Enforcement: Different roles such as SOC Analyst, Incident Responder, and Admin were defined. Each role had specific permissions determining which parts of the system they could access.

- OAuth2 / OpenID Connect Integration: Tools like TheHive and Shuffle were configured to delegate authentication to Keycloak using standard OAuth2 flows, ensuring secure token-based access.
- Secure Access to Sensitive Endpoints: The IP blocking API and SOAR workflow triggers were protected behind Keycloak-authenticated sessions, preventing unauthorized triggering of response actions.

The Keycloak setup ensured that the SOC platform was not just functional but also secure — only authorized personnel with the right roles could perform sensitive operations.

8. Dashboard Monitoring

Throughout the hackathon, dashboards played a critical role in maintaining visibility into the state of the environment. The following monitoring surfaces were active:

- Wazuh Threat Hunting Dashboard: Used to search and filter alerts in real time. The DQL-based query interface allowed quick lookups by rule ID, agent name, and time window. Both rule 100460 and 100470 were verified through this interface.
- Shuffle SOAR Execution History: Provided a live view of how many workflow runs had been completed, how many succeeded, and whether any had errors. This was essential for validating that automated responses were actually firing.
- TheHive Case Dashboard: Showed the current status of all open cases, including those linked to the sentrix organisation. Helped track the investigation lifecycle from alert intake to closure.
- Terminal / Server Logs: The Flask server running on port 8000 provided raw HTTP access logs, which were cross-checked against SOAR run timestamps to confirm end-to-end response accuracy.

Taken together, these dashboards gave the team a comprehensive, real-time view of the entire SOC pipeline during the attack simulation.

9. Brownie Points Implementation

Beyond the core requirements, the team implemented several advanced features that significantly enhanced the overall capability of the SOC platform. These additions are described below:

9.1 Custom Wazuh Detection Rules

- Two completely custom Wazuh rules (100460 and 100470) were written from scratch to address specific attack patterns.
- Rule 100460 performs multi-event correlation to detect attack campaigns, going beyond simple log matching.
- Rule 100470 implements behavioral deviation detection, which monitors for anomalies relative to a baseline — a technique used in advanced threat hunting.
- Both rules were validated and confirmed to fire correctly during the live simulation, with rule levels 12 and 13 respectively, indicating high-severity detections.

9.2 Real-Time Automated IP Blocking

- A lightweight Flask-based IP blocking service was developed and deployed on the local network.
- The service exposes a POST /block_ip endpoint that accepts an IP address and adds it to the block list immediately.
- This service was connected to Shuffle SOAR, enabling fully automated, zero-latency IP blocking in response to Wazuh alerts.
- The integration was tested under load — as shown in the terminal logs, dozens of block requests were processed within a two-minute window without errors.

9.3 Multi-Tool Integration (Wazuh + Shuffle + TheHive + Keycloak)

- Achieving bidirectional integration between four different tools (Wazuh, Shuffle, TheHive, Keycloak) in a hackathon environment is non-trivial.
- Alerts flowed from Wazuh into TheHive as cases and simultaneously triggered Shuffle workflows — without any manual steps.
- Keycloak provided authentication across the stack, making the entire pipeline enterprise-ready in terms of access control.

9.4 Behavioral Anomaly Detection

- Rather than relying only on signature-based detection, the team implemented an anomaly detection layer (Rule 100470) that can identify threats even if they don’t match known patterns.
- This is especially useful for detecting zero-day attacks or novel malware behavior that hasn’t been seen before.

Why These Improvements Matter

Together, these enhancements transform the project from a basic SIEM setup into a genuinely functional SOC platform. The combination of custom correlation rules, real-time automated response, and multi-tool integration demonstrates the kind of depth that real-world SOC teams operate with. The anomaly detection layer, in particular, shows an understanding of advanced threat detection concepts beyond what was explicitly required.

10. Final Results & Findings

By the end of Day 4, the following outcomes had been achieved and verified:

- Wazuh successfully generated high-severity alerts (levels 12 and 13) from two custom correlation rules during a live attack simulation.
- Suricata detected the high-frequency POST request flood on port 8000, confirming that network-level intrusion detection was functioning correctly.
- Shuffle SOAR completed multiple successful workflow executions, each resulting in a confirmed IP block (HTTP 200 “Blocked” response from the Flask endpoint).
- TheHive was operational with two organisations configured and ready to receive and track incidents from the alert pipeline.
- The full end-to-end flow — from attack traffic generation to automated IP block confirmation — was demonstrated successfully.

The system behaved as expected under realistic attack conditions. No false negatives were observed for the simulated attack patterns, and the automated response was consistently triggered within seconds of alert generation.

11. Challenges Faced

Like any real SOC deployment, the project came with its fair share of challenges:

- Rule Tuning: Getting the custom Wazuh rules to fire on exactly the right conditions without generating excessive false positives required multiple iterations of testing and refinement.
- Network Connectivity: Ensuring that all components (Wazuh, Shuffle, TheHive, Flask service) could communicate with each other in the isolated lab environment involved careful network configuration.

- Timing Synchronization: With automated workflows firing at high speed, ensuring that the Shuffle SOAR runs were mapped correctly to specific Wazuh alerts required careful logging and verification.
- Keycloak Integration Complexity: Configuring OAuth2 flows correctly for multiple downstream services took considerable time, especially for TheHive's SSO integration.
- Flask Service Stability: The IP blocking service needed to handle concurrent requests gracefully. Early versions would occasionally time out under load, which was resolved by optimizing the request handling logic.

12. Conclusion

Day 4 of the Mini Sentrix Hackathon brought the entire SOC pipeline together into a working, end-to-end system. The team successfully implemented threat detection at both the network level (Suricata) and the SIEM level (Wazuh), with custom correlation rules that go beyond basic out-of-the-box detection.

The automated response component, built using Shuffle SOAR and a custom Flask service, proved that the system could react to threats in real time without requiring manual analyst intervention. TheHive provided the investigation and case management layer, while Keycloak ensured that access to all components was properly secured.

The brownie point implementations — particularly the behavioral anomaly detection rule and the multi-tool integration — demonstrate a genuine understanding of how enterprise SOC platforms operate. The team did not just meet the requirements; it built something that mirrors real-world security operations center architecture.

Overall, Day 4 was a successful demonstration of a fully integrated, automated, and detection-capable mini SOC platform.